

Dokumentácia projektu Virtual Intersection

Táto dokumentácia sumarizuje celý projekt nachádzajúci sa v tomto priečinku. Vychádza z README súborov, zdrojového kódu, Docker/Kubernetes konfigurácií, OMNeT nastavení, tímového webu a pomocných súborov v jednotlivých moduloch.

Projekt **Virtual Intersection** je koordinovaný systém pre autonómne alebo virtuálne vozidlá prechádzajúce križovatkou. Aktuálny stav projektu sa sústreďuje hlavne na simulačnú časť: SUMO vytvára dopravnú simuláciu, OMNeT++ simuluje sieťovú komunikáciu, alg-runner počíta riadiace inštrukcie a central-unit koordinuje tok dát medzi službami. Reálne RC autá zatiaľ nie sú implementované.

Obsah

- [Cieľ projektu](#)
- [Aktuálny stav a rozsah](#)
- [Repozitárová štruktúra](#)
- [Architektúra](#)
- [Hlavný simulačný workflow](#)
- [Moduly](#)
- [API a dátové modely](#)
- [Frontend](#)
- [SUMO, SUMO API a SUMO service](#)
- [Central Unit](#)
- [Alg-runner](#)
- [OMNeT a OMNeT API](#)
- [Kubernetes nasadenie](#)
- [Lokálne spustenie](#)
- [CI/CD](#)
- [Tímový web a projektová história](#)
- [Prevádzkové poznámky a troubleshooting](#)
- [Známe limity a odporúčania](#)

Cieľ projektu

Cieľom projektu je navrhnuť a implementovať inteligentný systém, ktorý umožňuje vozidlám bezpečne a efektívne prechádzať križovatkou. Vozidlá majú komunikovať so systémom, systém má vyhodnocovať ich aktuálnu polohu, rýchlosť, smer a stav križovatky a následne posilať späť riadiace inštrukcie.

Projekt nadväzuje na predošlé tímové projekty a súvisiace práce, hlavne v oblastiach:

- simulácia dopravy cez SUMO,
- algoritmicke riadenie vozidiel,
- komunikácia cez OMNeT++,
- nasadenie služieb v Kubernetes,
- vizualizácia simulácie vo webovom rozhraní,
- budúca integrácia reálnych RC áut.

Aktuálny stav a rozsah

Aktuálna implementácia je simulačný základ pre ďalší vývoj. Hlavné hotové alebo rozpracované časti sú:

- webový frontend v Svelte/SvelteKit,
- SUMO API na upload konfigurácií a spúšťanie simulácií,
- SUMO service ako samostatný HTTP wrapper nad SUMO binárkou,
- central-unit ako orchestrátor medzi SUMO API, alg-runnerom a OMNeT,
- alg-runner s FIFO a priority/prioq algoritmom,
- OMNeT++ konfigurácia pre sieťovú simuláciu,
- OMNeT API na lifecycle správu OMNeT procesu,
- Kubernetes manifesty pre vývojové nasadenie,
- Dockerfile a docker-compose konfigurácie pre služby,
- tímový web s priebežnými zápisnicami.

Dôležité obmedzenia:

- reálne autá zatiaľ nie sú zapojené,
- chýba samostatný car-integration modul pre komunikáciu s fyzickými vozidlami,
- lokálne spustenie celého systému je obmedzené dostupnosťou OMNeT servera,
- aktuálny dátový tok virtuálnych áut ide cez SUMO -> central-unit -> OMNeT -> central-unit -> alg-runner -> central-unit -> OMNeT -> central-unit -> SUMO,
- OMNeT UDP most má passthrough fallback: ak OMNeT nie je dostupný, central-unit vracia pôvodné dáta ďalej, aby simulácia nespadla okamžite.

Repozitárová štruktúra

Koreňový priečinok obsahuje viac modulov, ktoré boli pôvodne samostatnými repozitármi:

Cesta	Účel
Onboarding/	Hlavné uvedenie do projektu, demo médiá, architektonické diagramy a docker-compose pre celý stack.
T7-team-website/	Jekyll tímový web so sprintmi, členmi tímu a zápisnicami.
.github/profile/README.md	Profil organizácie s popisom projektu, tímom a odkazmi.
frontend/	Svelte/SvelteKit používateľské rozhranie pre upload máp, ovládanie OMNeT, spustenie simulácie a replay.
central-unit/	FastAPI orchestrátor pre SUMO, alg-runner a OMNeT.
alg-runner/	FastAPI služba s algoritmi riadenia vozidiel.
sumo-api/	FastAPI API nad SUMO/TraCI, správa konfigurácií a ukladanie výsledkov simulácie.
sumo-service/	Ľahký HTTP server, ktorý štartuje a zastavuje SUMO binárku a vystavuje TraCI port.

Cesta	Účel
<code>sumo/</code>	Fork/upstream zdrojový strom Eclipse SUMO. Projekt bol testovaný s touto konkrétnou verziou.
<code>OMNET/</code>	OMNeT++ <code>.ned</code> , <code>.ini</code> , IP konfigurácia a dokumentácia sieťového nastavenia.
<code>omnet-api/</code>	FastAPI lifecycle API pre spúšťanie, zastavovanie a monitorovanie OMNeT++ na Ubuntu hoste.
<code>Kubernetes/</code>	Manifesty pre nasadenie do Kubernetes namespace <code>virtual-intersection-dev</code> .
<code>Virtual-Intersection.pptx</code>	Prezentačný materiál projektu.
<code>github.png</code> , <code>image-5.png</code> , <code>simulation-start.gif</code>	Vizuálne materiály/demá.

Adresár `sumo/` obsahuje veľký upstream projekt Eclipse SUMO vrátane vlastnej dokumentácie, zdrojových súborov, nástrojov, XSD schém a licencií. V tejto dokumentácii je SUMO popísané z pohľadu jeho použitia v tomto projekte.

Architektúra

Systém je distribuovaný stack služieb:

```

Frontend
|
| HTTP/HTTPS
v
Ingress / Central Unit / SUMO API proxy
|
+--> SUMO API <--> SUMO service <--> SUMO + TraCI
|
+--> Central Unit <--> alg-runner
|
+--> Central Unit <--> OMNeT UDP bridge
|
+--> Central Unit <--> OMNeT API REST proxy

```

Hlavné zodpovednosti:

- **Frontend** poskytuje používateľské rozhranie.
- **SUMO** simuluje cestnú sieť, vozidlá a ich pohyb.
- **SUMO service** štartuje/zastavuje SUMO proces a vystavuje TraCI port.
- **SUMO API** nahráva konfigurácie, spúšťa SUMO simuláciu, zbiera stav vozidiel cez TraCI a ukladá výstupy.
- **Central Unit** je stred systému: prijíma dáta z SUMO API, posiela ich cez OMNeT, volá alg-runner a vracia inštrukcie späť.

- **Alg-runner** počíta cieľové rýchlosti a riadiace inštrukcie.
- **OMNeT++** simuluje komunikačnú vrstvu.
- **OMNeT API** spravuje životný cyklus OMNeT++ procesu na hoste.
- **Kubernetes** orchestruje nasadenie služieb v clustri.

Hlavný simulačný workflow

1. Inicializácia služieb

Po nasadení cez Docker Compose alebo Kubernetes čakajú služby na požiadavky:

- frontend beží na porte **5173**,
- alg-runner na **8000**,
- central-unit na **8001**,
- sumo-api na **8002**,
- sumo-service na **8003** a TraCI na **1337**,
- OMNeT API typicky cez port **80** na hoste **192.168.20.51**,
- OMNeT UDP echo/bridge používa port **9999**.

2. Upload a načítanie SUMO konfigurácie

Používateľ vo frontende nahrá priečinok s trojicou súborov:

- ***.net.xml**,
- ***.rou.xml**,
- ***.sumocfg**.

SUMO API uloží súbory do **sumo-api/data/uploads/<uuid>/**. Frontend potom vie získať **net.xml**, parsovať sieť a vykresliť mapu na canvas.

Ak nie sú k dispozícii žiadne uložené konfigurácie, frontend má fallback test dáta v:

- **frontend/static/test-data/__testconfig/test.net.xml**,
- **frontend/static/test-data/__testconfig/test.rou.xml**,
- **frontend/static/test-data/__testconfig/test.sumocfg**.

3. Spustenie OMNeT++

Frontend má sekciu OMNeT settings, ktorá cez central-unit proxy volá OMNeT API:

- start,
- stop,
- restart,
- status,
- logs.

Pri štarte je možné poslať aj sieťové podmienky:

- bandwidth v Mbps,
- latency v ms,
- packet loss rate,

- bit error rate.

OMNeT API z nich vytvorí runtime `.ini` súbor s override nastaveniami.

4. Spustenie SUMO simulácie

Frontend vytvorí simuláciu cez `/sumo/api/v1/simulations`. SUMO API:

1. nájde `.sumocfg` v uloženom config priečinku,
2. zavolá SUMO service endpoint `/start`,
3. pripojí sa k SUMO cez TraCI,
4. vytiahne zo SUMO križovatky a hrany,
5. zaregistruje križovatky v central-unit,
6. vstúpi do krokového simulačného cyklu.

5. V2I štýl hlavného cyklu

V každom kroku simulácie:

1. SUMO API získa zo SUMO aktuálne vozidlá.
2. Pre každé vozidlo pripraví samostatnú správu s polohou, rýchlosťou, akceleráciou, cestou, pruhom a najbližšou križovatkou.
3. SUMO API posiela autá paralelne do central-unit endpointu `/sumo/car`.
4. Central-unit každú správu pošle cez OMNeT UDP klienta.
5. OMNeT vráti správu späť alebo central-unit použije passthrough.
6. Po odoslaní všetkých áut SUMO API zavolá `/sumo/step-complete`.
7. Central-unit pošle nazbierané dáta do alg-runnera na `/dispatch`.
8. Alg-runner vypočíta inštrukcie.
9. Central-unit pošle inštrukcie opäť cez OMNeT smerom späť.
10. SUMO API aplikuje prijaté rýchlosti cez TraCI volaním `traci.vehicle.setSpeed`.
11. SUMO vykoná ďalší simulačný krok.

Zjednodušene:

```
SUMO
-> SUMO service
-> SUMO API
-> Central Unit
-> OMNeT++
-> Central Unit
-> alg-runner
-> Central Unit
-> OMNeT++
-> Central Unit
-> SUMO API
-> SUMO
```

6. Výsledky simulácie

SUMO API ukladá:

- `simulation.json` s krokmi a vozidlami,
- `statistics.json` so štatistikami,
- central-unit zároveň zapisuje step logy do `central-unit/data/step_logs/<module_id>.jsonl`.

Frontend vie tieto dáta prehrať na canvas mape a zobrazíť štatistiky.

Moduly

Onboarding

`Onboarding/` je vstupný bod pre budúce tímy. Obsahuje:

- vysvetlenie rozsahu projektu,
- demo video a GIF ukážky,
- popis prístupov,
- architektúru,
- workflow pre reálne a virtuálne autá,
- docker-compose pre lokálny stack,
- diagramy v `Onboarding/diagram/`.

Prístupy podľa README:

- jump server a Kubernetes cluster rieši zodpovedná osoba alebo Ing. Matej Janeba PhD.,
- OMNeT server a RDP heslá rieši školiť alebo správca,
- GitHub prístupy vyžadujú pozvanie do organizácie,
- GHCR vyžaduje GitHub personal access token.

T7-team-website

`T7-team-website/` je Jekyll web s remote témou `jekyll/minima` a tmavým skinom. Web je dostupný na:

```
https://tp-2025-26-t7.github.io/T7-team-website/
```

Obsahuje:

- názov projektu: Virtuálna križovatka,
- vedúceho projektu: Ing. Jozef Juraško,
- členov tímu,
- sprinty,
- tímové a vedúcovské zápisnice.

Členovia tímu:

Meno	GitHub
Dominik Hajko	@XDhajko
Martin Horský	@MartinH2k3
Samuel Gabriel Galgóci	@SamoGG
Maryna Kolesnykova	@maryna0107
Bruno Kristián	@Brunokristi
Anna Skosar	@annaskosar

Sprintové ciele zachytené na webe:

- Sprint 1: organizácia práce, komunikácia, pochopenie zadania.
- Sprint 2: analýza existujúcich častí a návrh štruktúry riešenia.
- Sprint 3: dokončenie analytickej fázy a úprava kódu pre spracovanie v CUM/Central Unit.
- Sprint 4: realizácia návrhov, algoritmy, OMNeT++ a serverové prostredie.
- Sprint 5: pokračovanie implementácie a prepájanie komponentov.

Zápisnice dokumentujú vývoj od štartu projektu cez analýzu ReCo/Stocars, návrh architektúry, Kubernetes, OMNeT++, SUMO komunikáciu, central-unit, algoritmy, Dockerizáciu, optimalizácie alg-runnera, UI úpravy, lokálne nasadenie a prípravu prezentácií.

[.github/profile](#)

Profil organizácie opisuje projekt ako koordinovaný systém pre autonómne vozidlá v pohybe. Uvádza:

- cieľ komunikácie a koordinácie vozidiel,
- tím 7,
- vedúceho Ing. Jozefa Juraška,
- odkazy na tímový web, onboarding a produkčný projekt.

Produkčný odkaz uvedený v profile:

```
https://virtual-intersection-prod.ail-lab.fiit.stuba.sk/
```

API a dátové modely

Autentifikácia

SUMO API vyžaduje header:

```
Authorization: Bearer <API_KEY>
```

Vývojový kľúč použitý v konfiguráciách je:

S03FWz06Tu

Tento kľúč je v repozitári použitý ako vývojový secret. Pre produkčné nasadenie by sa mal považovať za citlivú hodnotu a rotovať mimo verejných manifestov.

SUMO API endpointy

Prefix: `/api/v1`

Metóda	Endpoint	Účel
GET	<code>/openapi.json</code>	OpenAPI schéma, ak je povolený Swagger.
POST	<code>/api/v1/config</code>	Upload <code>net.xml</code> , <code>rou.xml</code> , <code>sumocfg</code> .
GET	<code>/api/v1/config?contents=false</code>	Zoznam konfigurácií.
GET	<code>/api/v1/config/{cfg_uuid}</code>	Detail konfigurácie.
GET	<code>/api/v1/config/{cfg_uuid}/net</code>	Vráti <code>*.net.xml</code> ako XML.
DELETE	<code>/api/v1/config/{cfg_uuid}</code>	Vymaže konfiguráciu, ak nemá simulácie.
POST	<code>/api/v1/simulations</code>	Spustí simuláciu pre konfiguráciu.
GET	<code>/api/v1/simulations?contents=false</code>	Zoznam simulácií.
GET	<code>/api/v1/simulations/{sim_id}</code>	Výstup simulácie.
GET	<code>/api/v1/simulations/{sim_id}/statistics</code>	Štatistiky simulácie.
DELETE	<code>/api/v1/simulations/{sim_id}</code>	Vymaže simuláciu.

POST `/api/v1/simulations` očakáva:

```
{
  "stepspeed": 0.05,
  "uuid": "config-uuid-v4",
  "moduleId": "module-uuid-v4"
}
```

Výsledkom je `combined_uuid` v tvare:

```
<config_uuid>_<simulation_uuid>
```

Central Unit endpointy

Central-unit má hlavné endpointy pod `/sumo`, proxy pre SUMO API pod `/sumo/api/v1` a OMNeT proxy pod `/omnet`.

Metóda	Endpoint	Účel
GET	<code>/sumo/health</code>	Health check central-unit.
GET	<code>/sumo/step-log/{module_id}</code>	Vráti JSONL log krokov pre modul.
POST	<code>/sumo/register-junctions</code>	Registrácia križovatiek na začiatku simulácie.
POST	<code>/sumo/car</code>	Prijatie jedného auta v jednom kroku.
POST	<code>/sumo/step-complete</code>	Ukončenie kroku a výpočet inštrukcií cez alg-runner.
POST	<code>/sumo/step</code>	Legacy all-in-one krok.
GET/POST/DELETE	<code>/sumo/api/v1/...</code>	Proxy na SUMO API.
POST	<code>/omnet/simulation/start</code>	Proxy na OMNeT API start.
POST	<code>/omnet/simulation/stop</code>	Proxy na OMNeT API stop.
GET	<code>/omnet/simulation/status</code>	Proxy na OMNeT status.
POST	<code>/omnet/simulation/restart</code>	Proxy na OMNeT restart.
GET	<code>/omnet/simulation/logs</code>	Proxy na OMNeT logy.
GET	<code>/omnet/simulation/health</code>	Proxy na OMNeT health.

Central-unit model auta:

```
{
  "car_id": "veh0",
  "x": 1.0,
  "y": 2.0,
  "speed": 5.0,
  "acceleration": 0.2,
  "next_junction_id": "A0",
  "next_junction_x": 10.0,
  "next_junction_y": 20.0,
  "lane": "edge_0",
  "road": "edge"
}
```

`/sumo/car` očakáva:

```
{
  "module_id": "00000000-0000-4000-8000-000000000001",
}
```

```

"step_id": 1,
"algorithm_name": "fifo",
"total_cars": 10,
"car": {
  "car_id": "veh0",
  "x": 1.0,
  "y": 2.0,
  "speed": 5.0,
  "acceleration": 0.2
}
}

```

/sumo/step-complete vracia:

```

{
  "output": [
    {
      "car_id": "veh0",
      "speed": 4.2,
      "acceleration": null
    }
  ]
}

```

Alg-runner endpointy

Metóda	Endpoint	Účel
GET	/	Health check: {"status": "ok"}.
POST	/setup	Nastaví križovatky, cesty, ciele vozidiel a hyperparametre.
POST	/dispatch	Vypočíta cieľové rýchlosti a natočenie kolies.

/setup používa:

```

{
  "junctions": [
    {
      "junction_id": "j1",
      "connected_roads_count": 4,
      "connected_roads_ids": ["r-north", "r-east", "r-south", "r-west"],
      "x": 0.0,
      "y": 0.0,
      "junction_size": 1.0,
      "polygon": [[0.0, 0.0]]
    }
  ],
  "roads": [
    {

```

```

        "id": "r-west",
        "polyline": [[-10.0, 0.0], [0.0, 0.0]],
        "recommended_speed": 8.0,
        "junction_start_id": null,
        "junction_end_id": "j1"
    }
],
"car_targets": {
    "veh0": "r-east"
},
"overwrite": true,
"slowdown_zone": 3.0,
"slowdown_rate": 0.3
}

```

/dispatch používá:

```

{
  "algorithm_name": "fifo",
  "cars": [
    {
      "car_id": "veh0",
      "x": -3.0,
      "y": 0.0,
      "speed": 5.0,
      "wheel_rotation": 0.0,
      "rotation": 0.0,
      "acceleration": 2.0,
      "breaking": 2.0,
      "road_id": "r-west",
      "target_road_id": "r-east"
    }
  ],
  "next_request_in_seconds": 0.2
}

```

Odpověď:

```

{
  "cars": [
    {
      "car_id": "veh0",
      "speed": 4.2,
      "wheel_rotation": 0.0
    }
  ]
}

```

Podporované názvy algoritmov v kóde:

- `fifo`,
- `priority`,
- `prioq`.

Ak príde neznámy názov, použije sa FIFO.

SUMO service endpoints

SUMO service je jednoduchý HTTP server v `sumo-service/server.py`.

Metóda	Endpoint	Účel
POST	<code>/start</code>	Spustí SUMO proces.
POST	<code>/stop</code>	Zastaví aktuálny SUMO proces.
GET	<code>/health</code>	Health check.
GET	<code>/status</code>	Stav procesu a TraCI port.

`/start` používa:

```
{
  "config_path": "/sumo-api/data/uploads/.../config.sumocfg",
  "step_length": 0.05
}
```

SUMO service povoľuje config cesty iba v adresároch z `SUMO_CONFIG_DIR`, predvolene:

```
/configs,/sumo-api/data
```

OMNeT API endpoints

Metóda	Endpoint	Účel
POST	<code>/simulation/start</code>	Spustí OMNeT++ simuláciu.
POST	<code>/simulation/stop</code>	Zastaví OMNeT++ proces.
GET	<code>/simulation/status</code>	Vráti PID, stav, príkaz, log a runtime ini.
POST	<code>/simulation/restart</code>	Stop + start.
GET	<code>/simulation/logs?lines=200</code>	Vráti posledné riadky logu.
GET	<code>/simulation/health</code>	Health check.

Štartovací payload:

```
{
  "config_name": "General",
  "network_conditions": {
    "bandwidth_mbps": 100.0,
    "latency_ms": 15.5,
    "packet_loss_rate": 0.01,
    "bit_error_rate": 0.0001
  }
}
```

OMNeT API používa `filelock`, `psutil` a stavový súbor, aby:

- zabránilo paralelnému štartu viacerých simulácií,
- po reštarte API vedelo zistiť, či OMNeT proces ešte beží,
- vedelo ukončiť celý process group cez `SIGTERM` a potom `SIGKILL`,
- uchovávalo logy v `/tmp/omnet_api_logs` alebo podľa konfigurácie.

Frontend

Frontend je Svelte/SvelteKit aplikácia v `frontend/`.

Technológie:

- Svelte 5,
- SvelteKit,
- TypeScript,
- Vite,
- Tailwind CSS,
- Monaco editor ako závislosť,
- canvas render mapy.

NPM skripty:

```
npm run dev
npm run build
npm run preview
npm run prod
npm run check
```

Docker:

- base image: `node:24-alpine3.20`,
- port: `5173`,
- produkčný command: `npm run prod`.

Hlavná stránka `frontend/src/routes/+page.svelte` obsahuje:

- ľavý sidebar,

- sekciu Upload map,
- sekciu OMNeT settings,
- sekciu Simulation settings,
- mapový panel **MapView**.

Upload máp

Komponent **SumoFiles.svelte** umožňuje:

- upload priečinka,
- vyhľadanie súborov **.net.xml**, **.rou.xml**, **.sumocfg**,
- upload do SUMO API,
- načítanie existujúcej konfigurácie,
- soft-delete konfigurácie iba z UI zoznamu cez **localStorage**,
- fallback upload test konfigurácie, ak nie je nič dostupné.

Frontend ukladá do **localStorage**:

- vybranú konfiguráciu,
- skryté konfigurácie,
- simulačné nastavenia,
- OMNeT nastavenia a logy.

Mapa a replay simulácie

MapView.svelte:

- parsuje SUMO **net.xml**,
- filtruje interné križovatky a interné hrany,
- kreslí cesty, pruhy, križovatky a vozidlá na canvas,
- podporuje zoom a pan,
- prehráva uloženú simuláciu krok po kroku,
- zobrazuje štatistiky,
- vie načítať mapu podľa UUID zo simulation id.

Vozidlá sa kreslia ako jednoduché obdĺžniky. Smer vozidla sa odhaduje podľa ďalšej polohy v nasledujúcom kroku.

Simulation settings

Modules.svelte obsahuje:

- výber algoritmu v UI,
- rozmery auta,
- step speed,
- kontrolu, či OMNeT beží,
- spustenie simulácie cez SUMO API.

Poznámka k aktuálnemu stavu: UI má výber algoritmu, ale simulačný tok v **sumo-api/app/utils/simulate.py** aktuálne posiela do central-unit **algorithm_name="fifo"**. Hodnota z

UI teda nie je plne zapojená do reálneho spustenia simulácie. Navyše UI používa hodnotu `priority_queue`, zatiaľ čo alg-runner registruje `priority` a `prioq`.

OMNeT settings

`Simulations.svelte` umožňuje:

- pridávať lokálne názvy OMNeT configov,
- štartovať, zastavovať a reštartovať OMNeT,
- získať status a logy,
- nastaviť sieťové podmienky,
- použiť presety `Balanced` a `Lossy link`.

Frontend env

`frontend/.env.example`:

```
PUBLIC_ALG_RUNNER_API="http://localhost:8002"
PUBLIC_SUMO_API="http://localhost:8001"
PUBLIC_SUMO_API_KEY="S03FWz06Tu"
```

V aktuálnom kóde klient používa hlavne relatívne cesty `/sumo`, `/alg`, `/omnet`, ktoré v clustri smeruje ingress alebo central-unit proxy.

SUMO, SUMO API a SUMO service

SUMO

`sumo/` obsahuje fork Eclipse SUMO. README explicitne upozorňuje, že aplikácia bola testovaná s týmto konkrétnym repozitárom a novšie verzie SUMO môžu meniť správanie.

SUMO je open-source mikroskopický dopravný simulátor. V tomto projekte poskytuje:

- cestnú sieť,
- križovatky,
- hrany/cesty,
- pruhy,
- vozidlá,
- krokovanie simulácie,
- TraCI rozhranie na riadenie počas behu.

SUMO má licenciu Eclipse Public License 2.0. Súbory `sumo/LICENSE`, `sumo/NOTICE.md`, `sumo/CODE_OF_CONDUCT.md` a `sumo/SECURITY.md` patria k upstream projektu.

SUMO API

SUMO API je FastAPI služba v `sumo-api/`. Hlavné súbory:

- `app/main.py` - aplikácia, CORS, API key middleware, routery,

- `app/routers/config.py` - upload/list/get/delete konfigurácií,
- `app/routers/simulate.py` - spustenie a čítanie simulácií,
- `app/utils/simulate.py` - SUMO štart, TraCI cyklus, komunikácia s central-unit,
- `app/utils/config.py` - načítanie config priečinkov,
- `app/utils/data.py` - dátové adresáre a UUID,
- `app/env.py` - settings cez `.env`.

Dátové adresáre:

```
data/uploads
data/simulations
data/logs
```

Simulačné štatistiky:

- `crashed_vehicles_count`,
- `real_duration`,
- `step_speed`,
- `simulated_duration`,
- `step_count`,
- `vehicle_count`,
- `module_name`,
- `avg_frame_computation_time`,
- `avg_waiting_time`.

SUMO API env:

Premenná	Účel
<code>API_KEY</code>	Bearer token pre API.
<code>CENTRAL_UNIT_HOST</code>	URL central-unit.
<code>FRONTEND_HOST</code>	CORS frontend host, môže byť <code>*</code> .
<code>SUMO_HOST</code>	TraCI host, napr. <code>http://127.0.0.1:1337</code> .
<code>SUMO_SERVER_URL</code>	SUMO service HTTP URL, napr. <code>http://127.0.0.1:8003</code> .
<code>PRINT_INFO_LOGS</code>	Či tlačíť info logy do stdout.
<code>EXPOSE_SWAGGER</code>	Či vystaviť <code>/docs</code> , <code>/redoc</code> , <code>/openapi.json</code> .
<code>ISOLATED_MODE</code>	Ak <code>True</code> , SUMO API nevolá central-unit.

Poznámka: `sumo-api/requirements.txt` je v pracovnom strome uložený v neštandardne vyzerajúcom kódovaní s nulovými bajtmi. Obsahovo uvádza FastAPI, uvicorn, pydantic, requests, httpx, TraCI, sumolib a ďalšie závislosti. Pred buildom je vhodné overiť, či ho `pip` v danom prostredí prečíta korektne.

SUMO service

SUMO service v `sumo-service/` je oddelený od SUMO API, aby:

- SUMO API nemuselo priamo spravovať binárku,
- v Kubernetese mohol SUMO bežať ako sidecar,
- TraCI komunikácia mohla ísť lokálne v rámci Podu.

Dockerfile:

- v builder stage klonuje Eclipse SUMO,
- vypína GUI/netedit/webwizard/testy/dokumentáciu,
- builduje potrebné binárky,
- runtime image obsahuje SUMO binárky, tools a `server.py`.

SUMO process sa spúšťa s argumentmi:

```
sumo
-c <config_path>
--start
--quit-on-end
--remote-port <TRACI_PORT>
--step-length <step_length>
--collision.action=warn
--collision.check-junctions=true
```

Central Unit

Central Unit je FastAPI služba v `central-unit/`. Je centrálnym orchestrátorom.

Hlavné súbory:

- `app/main.py` - FastAPI app, lifecycle, CORS, routery,
- `app/api/sumo_api.py` - SUMO-facing endpoints a komunikácia s alg-runnerom,
- `app/api/sumo_proxy.py` - proxy na SUMO API,
- `app/api/omnet_proxy.py` - proxy na OMNeT API,
- `app/helpers/omnet_socket.py` - UDP klient pre OMNeT bridge.

Startup:

- pokúsi sa vytvoriť UDP endpoint na OMNeT,
- vytvorí persistentný `httpx.AsyncClient`,
- na shutdown zavrie OMNeT klienta a HTTP klienta.

Central-unit env:

Premenná	Predvolená hodnota / účel
<code>ALG_RUNNER_URL</code>	<code>http://alg-runner:8000</code> alebo lokálne <code>http://localhost:8000</code> .
<code>SUMO_API_URL</code>	<code>http://sumo-api:8002</code> .

Premenná	Predvolená hodnota / účel
OMNET_API_URL	http://192.168.20.51:80.
OMNET_API_FALLBACK_URL	fallback OMNeT API URL.
OMNET_HOST	192.168.20.51.
OMNET_PORT	9999.
OMNET_BYPASS_TIMEOUT	timeout pre OMNeT odpoveď, v kóde default 2.
ALG_RUNNER_DEFAULT_ROAD_SPEED_MPS	13.9.
OMNET_CONNECT_TIMEOUT	15.
OMNET_WRITE_TIMEOUT	30.
OMNET_READ_TIMEOUT	120.
OMNET_MAX_UDP_PAYLOAD	1450.
OMNET_FRAGMENT_TTL_SECONDS	10.

OMNeT UDP klient

omnet_socket.py:

- posiela JSON cez UDP,
- používa `asyncio.Lock`, aby sa nemiešali odpovede,
- podporuje fragmentáciu väčších payloadov cez prefix `FRAG1|`,
- rekonštruuje fragmenty podľa message id,
- pri timeoute alebo chybe prejde do passthrough režimu,
- passthrough znamená, že vráti rovnaké dáta, aké dostal.

Tento fallback je dôležitý pre vývoj a stabilitu, ale môže maskovať skutočnosť, že OMNeT nie je zapojený do dátového toku.

Step logy

Central-unit zapisuje JSONL log do:

```
central-unit/data/step_logs/<module_id>.jsonl
```

Každý záznam obsahuje:

- timestamp,
- module_id,
- trvanie výpočtu,
- vstupný payload,
- výsledné inštrukcie.

Alg-runner

Alg-runner je FastAPI služba v `alg-runner/`, ktorá počíta riadiace inštrukcie.

Hlavné súbory:

- `app/main.py` - app state a router,
- `app/routes/algorithms.py` - `/setup`, `/dispatch`,
- `app/models/schema.py` - Road, Junction, Car, CarDispatchTarget,
- `app/models/road_network.py` - pomocné výpočty nad cestnou sieťou,
- `app/algorithms/fifo.py` - FIFO algoritmus,
- `app/algorithms/prioq.py` - prioritný algoritmus,
- `tests/test_dispatch_api.py` - API testy pre setup, dispatch, brzdenie, rozostupy a prioritu.

Závislosti:

- FastAPI,
- uvicorn,
- shapely,
- orjson,
- pydantic,
- dotenv,
- requests/httpx pre testy.

FIFO algoritmus

FIFO výpočet:

- nájde cestu auta podľa `road_id`, `target_road_id` alebo najbližšej cesty,
- nájde najbližšiu alebo explicitnú križovatku,
- spočíta vzdialenosť ku križovatke,
- aplikuje slowdown zone,
- vyberie prioritné auto v každej križovatkovej skupine podľa vzdialenosti,
- neprioritným autám v control zone nastaví limit tak, aby zastavili pred križovatkou,
- v rámci rovnakej cesty obmedzí zadné auto podľa rozostupu od predného auta,
- vypočíta cieľovú rýchlosť pre nasledujúci interval.

Dôležité konštanty:

- follow base gap: `2.0`,
- follow time headway: `1.2`,
- junction stop buffer: `1.0`,
- min duration: `1e-3`.

Ak `breaking` nie je poslané, algoritmus používa opačnú hodnotu `acceleration` ako brzdenie.

Priority/prioq algoritmus

`priority` a `prioq` sú aliasy na rovnaký algoritmus. Ten:

- zistí pre auto vstupnú cestu,

- zistí cieľovú cestu,
- odhadne počet segmentov križovatky, ktoré auto obsadí,
- autá s kratšou obsadenosťou segmentov dostávajú vyššiu prioritu,
- samotný výpočet rýchlostí deleguje na FIFO cez vlastnú `priority_key_fn`.

V testoch je overené, že pravé odbočenie s kratšou obsadenosťou segmentov dostane prioritu pred ľavým odbočením.

OMNeT a OMNeT API

OMNeT simulácia

OMNET/ obsahuje konfiguráciu pre OMNeT++ a INET framework.

Hlavné súbory:

- `OMNET/ExtPingNet.ned`,
- `OMNET/omnetpp.ini`,
- `OMNET/config.xml`,
- `OMNET/images/arch.png`,
- `OMNET/README.md`.

Sieť `ExtPingNet` obsahuje:

- `Ipv4NetworkConfigurator`,
- `StandardHost` s jedným Ethernet rozhraním,
- povolené nepripojené spojenia.

`omnetpp.ini`:

- používa headless `Cmdenv`,
- vypína event bannery,
- používa `inet::RealTimeScheduler`,
- nastavuje NED path na INET a Simu5G,
- viaže `host1.eth[0]` na OS interface `veth-sim`,
- nastavuje `UdpEchoApp` na porte `9999`,
- používa vypočítané CRC/FCS/checksum režimy.

`config.xml` dáva simulovanému rozhraniu:

```
<interface hosts="host1" names="eth*" address="10.255.0.2"
netmask="255.255.255.252"/>
```

Prístup na OMNeT server

Podľa README:

1. Získať prístup na jump server.
2. Vytvoriť SSH tunel:

```
ssh -L 3367:192.168.20.51:3389 "USERNAME"@ail-lab.fiit.stuba.sk -p 8225
```

3. Pripojiť sa cez Remote Desktop na:

```
localhost:3367
```

Používateľ servera je **tp7**. Heslá treba riešiť so školiteľom alebo správcom.

Spustenie OMNeT na Linux serveri

Na serveri má byť alias:

```
omnet
```

Po nastavení prostredia:

```
omnetpp
```

Aj keď príkaz znie GUI-ovo, konfigurácia používa headless **Cmdenv**.

OMNeT sieťové rozhrania a recovery

OMNeT server používa **veth-host** a **veth-sim**.

Očakávaný stav:

- **veth-host** má IP **10.255.0.1/24**,
- **veth-sim** nemá IP adresu,
- simulovaný host v OMNeT má **10.255.0.2**,
- route existuje pre **10.255.0.2/32** dev **veth-host**,
- UDP port **9999** je smerovaný na simuláciu.

Ak server zlyhá, README odporúča obnoviť veth pár, sysctl a iptables pravidlá.

Kontrola:

```
ip -br addr show dev veth-host
ip -br addr show dev veth-sim
ip route | grep 10.255.0.2
sudo iptables -t nat -nvL PREROUTING
sudo iptables -nvL FORWARD
```

Monitorovanie UDP prevádzky:

```
sudo tcpdump -ni ens6np0 udp port 9999
sudo tcpdump -ni veth-host udp port 9999
sudo tcpdump -ni veth-sim udp port 9999
```

Ping/test packet:

```
echo -n '{"type":"ping"}' | nc -u -w1 192.168.20.51 9999
```

OMNeT API

`omnet-api/` poskytuje REST lifecycle manažment OMNeT simulácie.

Hlavné súbory:

- `app/main.py`,
- `app/routes/simulation.py`,
- `app/services/simulation_manager.py`,
- `app/services/process_store.py`,
- `app/services/config_builder.py`,
- `app/config.py`,
- `app/schemas.py`,
- `nginx.conf`,
- `start.sh`.

Konfigurácia:

Premenná	Predvolená hodnota
<code>BASE_DIR</code>	<code>/home/tp7/Desktop/omnet/rcproject</code>
<code>SRC_DIR</code>	<code>/home/tp7/Desktop/omnet/rcproject/src</code>
<code>BINARY_NAME</code>	<code>rcproject</code>
<code>BASE_INI_FILE</code>	<code>../simulations/omnetpp.ini</code>
<code>OPP_VENV_ACTIVATE</code>	<code>/home/tp7/Desktop/opp_env_venv/bin/activate</code>
<code>STATE_FILE</code>	<code>/tmp/omnet_simulation_state.json</code>
<code>LOCK_FILE</code>	<code>/tmp/omnet_simulation.lock</code>
<code>LOG_DIR</code>	<code>/tmp/omnet_api_logs</code>
<code>SIGTERM_TIMEOUT_SEC</code>	<code>10</code>

Dockerfile:

- používa `python:3.11-slim`,
- inštaluje `procps` a `nginx`,
- uvicorn beží lokálne na `127.0.0.1:8000`,
- NGINX vystavuje port `80`.

`start.sh`:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000 &  
exec nginx -g "daemon off;"
```

Kubernetes nasadenie

Manifesty sú v `Kubernetes/` a cieľia na namespace:

```
virtual-intersection-dev
```

Vývojový host:

```
virtual-intersection-dev.ail-lab.fiit.stuba.sk
```

Produkčný namespace v dokumentácii:

```
virtual-intersection-prod
```

Produkčný host:

```
virtual-intersection.ail-lab.fiit.stuba.sk
```

Súbory a poradie

Číselné prefixy určujú poradie aplikovania:

- `10–19`: ConfigMapy a Secret,
- `20–29`: Deploymenty,
- `30–39`: Service,
- `40–49`: Ingress,
- `50–59`: NetworkPolicy.

Zoznam:

```

10-sumo-api-config.yaml
11-sumo-api-secret.yaml
12-central-unit-config.yaml
13-frontend-config.yaml
14-alg-runner-config.yaml
20-alg-runner-deploy.yaml
21-central-unit-deploy.yaml
22-sumo-api-deploy.yaml
23-frontend-deploy.yaml
30-alg-runner-svc.yaml
31-central-unit-svc.yaml
32-sumo-api-svc.yaml
33-frontend-svc.yaml
40-ingress.yaml
50-egress-omnet.yaml

```

Deployments

Deployment	Image	Port	Poznámka
alg-runner	ghcr.io/tp-2025-26-t7/alg-runner:latest	8000	Health na /.
central-unit	ghcr.io/tp-2025-26-t7/central-unit:latest	8001	Health na /sumo/health, data volume emptyDir.
sumo-api	ghcr.io/tp-2025-26-t7/sumo-api:latest	8002	Beží v jednom pode so SUMO sidecarom.
sumo sidecar	ghcr.io/tp-2025-26-t7/sumo-service:latest	8003, 1337	HTTP + TraCI.
frontend	ghcr.io/tp-2025-26-t7/frontend:latest	5173	Web UI.

SUMO sidecar pattern

Kubernetes používa pre SUMO API multi-container pod:

```

sumo-api pod
|- sumo-api container :8002
|- sumo sidecar       :8003, :1337
|- shared emptyDir    /sumo-api/data

```

Výhody:

- TraCI ide cez localhost,
- configy sú zdieľané cez emptyDir,
- nevzniká inter-pod latency medzi SUMO API a SUMO,

- zodpovednosti sú oddelené.

Ingress

Ingress host:

```
virtual-intersection-dev.ail-lab.fiit.stuba.sk
```

Routovanie:

Path	Service	Port
/	frontend	5173
/sumo/*	sumo-api	8002
/central/*	central-unit	8001
/alg/*	alg-runner	8000

Ingress používa NGINX regex a rewrite target `/$2`.

NetworkPolicy

`50-egress-omnet.yaml` je dočasne permissive:

```
egress:
- {}
```

To umožňuje komunikáciu v namespace, DNS aj externý prístup na OMNeT host. Pre produkciu je odporúčané obmedziť egress na konkrétne služby, DNS a OMNeT host/porty.

Nasadenie

Vývojové nasadenie:

```
kubectl apply -f . --recursive -n virtual-intersection-dev
```

Overenie:

```
kubectl get pods -n virtual-intersection-dev
kubectl get svc -n virtual-intersection-dev
kubectl get ingress -n virtual-intersection-dev
```

Rollout restart:

```
kubectl rollout restart deployment/sumo-api -n virtual-intersection-dev
kubectl rollout restart deployment/central-unit -n virtual-intersection-dev
kubectl rollout restart deployment/alg-runner -n virtual-intersection-dev
kubectl rollout restart deployment/frontend -n virtual-intersection-dev
```

Logy:

```
kubectl logs -n virtual-intersection-dev deployment/sumo-api -c sumo-api
kubectl logs -n virtual-intersection-dev deployment/sumo-api -c sumo
kubectl logs -n virtual-intersection-dev deployment/central-unit
kubectl logs -n virtual-intersection-dev deployment/alg-runner
```

Produkčné nasadenie má byť manuálne, až po otestovaní vo vývojovom prostredí.

Lokálne spustenie

Celý stack cez Onboarding docker-compose

V koreňovom priečinku so službami:

```
docker compose up --build
```

Porty:

Služba	Port
frontend	5173
alg-runner	8000
central-unit	8001
sumo-api	8002
sumo-service HTTP	8003
TraCI	1337

Poznámka: README upozorňuje, že aktuálna verzia nie je plne lokálne samostatná, pretože vyžaduje OMNeT server. Lokálne compose smeruje OMNeT na externý host **192.168.20.51**.

Samostatné služby

Alg-runner:

```
cd alg-runner
pip install -r requirements.txt
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Central Unit:

```
cd central-unit
docker-compose up --build -d
```

SUMO API:

```
cd sumo-api
docker compose up --build
```

Frontend:

```
cd frontend
npm install
npm run dev
```

OMNeT API:

```
cd omnet-api
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

SUMO service:

```
cd sumo-service
docker build -t sumo-service .
docker run -p 8003:8003 -p 1337:1337 sumo-service
```

CI/CD

Jednotlivé moduly majú vlastné GitHub Actions workflowy, ktoré buildujú Docker image do GHCR. Väčšina služieb potom používa self-hosted runner na rollout do MicroK8s vývojového namespace.

Modul	Workflow	Vetva	Image	Deploy
alg-runner	.github/workflows/ghcr.yml	master	ghcr.io/tp-2025-26-t7/alg-runner	deployment alg-runner.

Modul	Workflow	Vetva	Image	Deploy
central-unit	.github/workflows/ghcr.yml	main	ghcr.io/tp-2025-26-t7/central-unit	deployment central-unit.
frontend	.github/workflows/ghcr.yml	main	ghcr.io/tp-2025-26-t7/frontend	deployment frontend.
sumo-api	.github/workflows/ghcr.yml	master	ghcr.io/tp-2025-26-t7/sumo-api	container sumo-api v deploymente sumo-api.
sumo-service	.github/workflows/ghcr.yml	main	ghcr.io/tp-2025-26-t7/sumo-service	container sumo v deploymente sumo-api.
omnet-api	.github/workflows/omnet-api-ghcr.yml	main	ghcr.io/tp-2025-26-t7/omnet-api	iba build/push podľa workflowu.

Tagy:

- [latest](#),
- [\\${{ github.sha }}](#).

Deploy príkazy používajú:

```
microk8s kubectl set image ...
microk8s kubectl rollout status ...
```

Tímový web a projektová história

Projektová história zo zápisníc:

- Úvod: dohodnuté dokumentovanie, verzovanie, preskúmanie ReCo a Stocars.
- Prvé týždne: definovanie cieľa, technológií, formátu dát zo SUMO do algoritmu a späť.
- Architektúra: vznik diagramov, diskusia komunikácie medzi modulmi, výber štruktúry repozitárov.
- Kubernetes: analýza požiadaviek, nasadenie, vývojové a produkčné prostredie.
- SUMO: skúmanie SUMO API, TraCI, dostupných dát o vozidlách, scenárov a metrík.
- Alg-runner: preklad legacy FIFO z C++ do Pythonu, pridávanie ďalších algoritmov, optimalizácie.
- Central Unit: prepájací modul, logovanie krokov, orchestrácia medzi simuláciou a algoritmom.
- OMNeT++: zavedenie sieťovej simulácie, požiadavky, serverové nastavenie, testovanie.
- Frontend: upload konfigurácií, vizualizácia mapy, replay, štatistiky, OMNeT ovládanie a sidebar nastavenia.

- Dockerizácia: služby dostali Dockerfile a compose konfigurácie.
- Druhý semester: zefektívňovanie systému, približovanie simulácie realite, lokálne nasadenie, stabilita komunikácie, príprava prezentácií.

Prevádzkové poznámky a troubleshooting

Kubernetes

Popis podu:

```
kubectl describe pod <pod-name> -n virtual-intersection-dev
```

DNS test:

```
kubectl run -it --rm debug --image=busybox --restart=Never -n virtual-intersection-dev -- nslookup central-unit
```

HTTP test:

```
kubectl run -it --rm debug --image=curlimages/curl --restart=Never -n virtual-intersection-dev -- curl http://sumo-api:8002
```

SUMO sidecar test:

```
kubectl exec -it deployment/sumo-api -c sumo-api -n virtual-intersection-dev -- /bin/sh  
curl http://127.0.0.1:8003
```

Resource usage:

```
kubectl top pods -n virtual-intersection-dev
```

OMNeT

Ak prichádzajú pakety na `ens6np0`, ale nie na `veth-sim`, problém je pravdepodobne v iptables alebo veth rozhraniach.

Ak OMNeT nie je dostupný, central-unit môže prejsť do passthrough režimu. Simulácia potom môže pokračovať, ale bez reálneho sieťového efektu OMNeT++.

SUMO

Ak SUMO API nevie spustiť simuláciu:

- skontrolovať, či `*.sumocfg` existuje v upload priečinku,
- skontrolovať, či `SUMO_SERVER_URL` smeruje na SUMO service,
- skontrolovať `/status` SUMO service,
- skontrolovať TraCI port `1337`,
- skontrolovať, či cesta ku configu je pod povoleným adresárom.

Frontend

Ak sa nezobrazuje mapa:

- overiť, či bola konfigurácia nahratá,
- overiť `GET /sumo/api/v1/config/{id}/net`,
- skontrolovať browser console,
- skontrolovať, či `PUBLIC_SUMO_API_KEY` sedí s API key v SUMO API.

Ak sa simulácia nedá spustiť:

- OMNeT musí bežať podľa kontroly vo frontende,
- musí byť načítaná konfigurácia,
- SUMO API musí byť dostupné cez `/sumo/api/v1/simulations`.

Známe limity a odporúčania

1. Reálne autá nie sú implementované. Treba doplniť car-integration modul a upraviť central-unit tak, aby vedel pracovať aj s trasovaním a telemetriou reálnych áut.
2. Výber algoritmu z frontendu nie je plne prepojený do SUMO simulačného toku. Treba posilať hodnotu z UI do SUMO API a ďalej do central-unit/alg-runnera.
3. Hodnota `priority_queue` vo frontende nezodpovedá registrovaným názvom `priority/prioq` v alg-runneri.
4. OMNeT UDP passthrough fallback je praktický pri vývoji, ale pri testovaní sieťových efektov treba explicitne overiť, či OMNeT odpovedá.
5. Kubernetes NetworkPolicy je aktuálne allow-all egress. Pre produkciu odporúčané sprísniť.
6. API key je v repozitári v manifestoch a `.env.example`. Pre produkciu treba rotovať a manažovať ho ako secret.
7. `sumo-api/requirements.txt` treba overiť kvôli neštandardnému kódovaniu.
8. `central-unit/Dockerfile` obsahuje duplicitný `CMD`, čo je neškodné, ale vhodné vyčistiť.
9. Lokálne spustenie nie je úplne nezávislé od infraštruktúry, pretože OMNeT beží na externom serveri.
10. OMNeT API mapovanie sieťových podmienok na INET wildcard parametre treba overiť podľa skutočných NED modulov.
11. `sumo/` je veľký upstream fork. Ak sa aktualizuje na novú verziu SUMO, treba regresne otestovať TraCI, formát siete, generovanie interných hrán a správanie simulácie.

Rýchly referenčný prehľad portov

Služba	Port
alg-runner	8000
central-unit	8001
sumo-api	8002
sumo-service HTTP	8003
SUMO TraCI	1337
frontend	5173
OMNeT API	typicky 80 na 192.168.20.51
OMNeT UDP bridge/echo	9999

Rýchly referenčný workflow pre používateľa

1. Otvoriť frontend.
2. Nahrať alebo vybrať SUMO konfiguráciu.
3. Načítať mapu.
4. V OMNeT settings spustiť OMNeT++.
5. V Simulation settings nastaviť step speed.
6. Spustiť simuláciu.
7. Po dokončení prehrať simuláciu na mape.
8. Skontrolovať štatistiky a logy.

Rýchly referenčný workflow pre vývojára

1. Prečítať [Onboarding/README.md](#).
2. Skontrolovať env premenné služieb.
3. Pre vývoj API spustiť relevantnú službu samostatne.
4. Pre integračné testovanie použiť vývojový Kubernetes namespace.
5. Pri zmene Docker image overiť GHCR workflow.
6. Pri problémoch s komunikáciou sledovať logy v poradí frontend -> SUMO API -> central-unit -> alg-runner/OMNeT -> SUMO service.
7. Pri algoritmoch spustiť alebo doplniť testy v [alg-runner/tests/test_dispatch_api.py](#).

Záver

Projekt Virtual Intersection tvorí funkčný základ pre vývoj inteligentného systému riadenia vozidiel na križovatke. Aktuálna implementácia úspešne prepája dopravnú simuláciu v SUMO, algoritmické rozhodovanie v alg-runneri, centrálnu orchestráciu v central-unit, sieťovú vrstvu cez OMNeT++ a používateľské rozhranie vo frontende. Vďaka Dockeru, Kubernetes manifestom a CI/CD workflowom je projekt pripravený na ďalší vývoj, testovanie a nasadzovanie vo vývojovom prostredí.

Najväčšou hodnotou tejto verzie je vytvorenie ucelenej simulačnej platformy, na ktorej môžu ďalšie tímy stavať. Systém už umožňuje nahrávať SUMO konfigurácie, spúšťať simulácie, sledovať stav vozidiel, posilať dáta cez central-unit a OMNeT cestu, počítat radiace inštrukcie a spätne vizualizovať výsledky.

Zároveň sú jasne pomenované miesta, ktoré treba doplniť: najmä integrácia reálnych RC áut, úplné prepojenie výberu algoritmu z frontendu, sprísnenie produkčnej infraštruktúry a ďalšie overovanie OMNeT komunikácie.

Dokumentácia by mala slúžiť ako orientačný aj praktický materiál pre budúcich vývojárov. Pri pokračovaní projektu je najdôležitejšie zachovať jasné hranice medzi modulmi, priebežne aktualizovať API kontrakty a každú zmenu v simulačnom toku overovať end-to-end, pretože výsledné správanie vzniká spoluprácou viacerých služieb naraz.